```
!pip install -U transformers
!pip install -U atcelerate
!pip install -U dataset
!pip istall -U bertviz
!pip install -U umap-learn
!pip install seaborn --upgrade
```

```
  Attempting uninstall: transformers
    Found existing installation: transformers 4.56.1
    Uninstalling transformers-4.56.1:
      Successfully uninstalled transformers-4.56.1
Successfully installed transformers-4.56.2
ERROR: Could not find a version that satisfies the requirement atcelerate (from vers
ERROR: No matching distribution found for atcelerate
Collecting dataset
  Downloading dataset-1.6.2-py2.py3-none-any.whl.metadata (1.9 kB)
Collecting sqlalchemy<2.0.0,>=1.3.2 (from dataset)
  Downloading SQLAlchemy-1.4.54-cp312-cp312-manylinux_2_5_x86_64.manylinux1_x86_64.m
Requirement already satisfied: alembic>=0.6.2 in /usr/local/lib/python3.12/dist-pack
Collecting banal>=1.0.1 (from dataset)
  Downloading banal-1.0.6-py2.py3-none-any.whl.metadata (1.4 kB)
Requirement already satisfied: Mako in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: typing-extensions>=4.12 in /usr/local/lib/python3.12/
Requirement already satisfied: greenlet!=0.4.17 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: MarkupSafe>=0.9.2 in /usr/local/lib/python3.12/dist-p
Downloading dataset-1.6.2-py2.py3-none-any.whl (18 kB)
Downloading banal-1.0.6-py2.py3-none-any.whl (6.1 kB)
Downloading SQLAlchemy-1.4.54-cp312-cp312-manylinux_2_5_x86_64.manylinux1_x86_64.mar
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.6/1.6 MB 34.4 MB/s eta 0:00:00
Installing collected packages: banal, sqlalchemy, dataset
  Attempting uninstall: sqlalchemy
    Found existing installation: SQLAlchemy 2.0.43
    Uninstalling SQLAlchemy-2.0.43:
      Successfully uninstalled SQLAlchemy-2.0.43
ERROR: pip's dependency resolver does not currently take into account all the packag
ipython-sql 0.5.0 requires sqlalchemy>=2.0, but you have sqlalchemy 1.4.54 which is
```

```
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dis
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (
```

```
import pandas as pd
url = "https://raw.githubusercontent.com/laxmimerit/All-CSV-ML-Data-Files-Download/m
df = pd.read_csv(url)
display(df.head(16000))
```

	text	label	label_name
0	i didnt feel humiliated	0	sadness
1	i can go from feeling so hopeless to so damned...	0	sadness
2	im grabbing a minute to post i feel greedy wrong	3	anger
3	i am ever feeling nostalgic about the fireplac...	2	love
4	i am feeling grouchy	3	anger
...	...	...	...
15995	i just had a very brief time in the beanbag an...	0	sadness
15996	i am now turning and i feel pathetic that i am...	0	sadness
15997	i feel strong and good overall	1	joy
15998	i feel like this was such a rude comment and i...	3	anger
15999	i know a lot but i feel so stupid because i ca...	0	sadness

16000 rows × 3 columns

```
df.info()
df.isnull().sum()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 16000 entries, 0 to 15999
Data columns (total 3 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   text        16000 non-null  object
 1   label       16000 non-null  int64
 2   label_name  16000 non-null  object
dtypes: int64(1), object(2)
memory usage: 375.1+ KB
```

	0
text	0
label	0
label_name	0

dtype: int64

```
df.describe()
## It tell about that is there is any int values
```

	label
count	16000.000000
mean	1.565937
std	1.501430
min	0.000000
25%	0.000000
50%	1.000000
75%	3.000000
max	5.000000

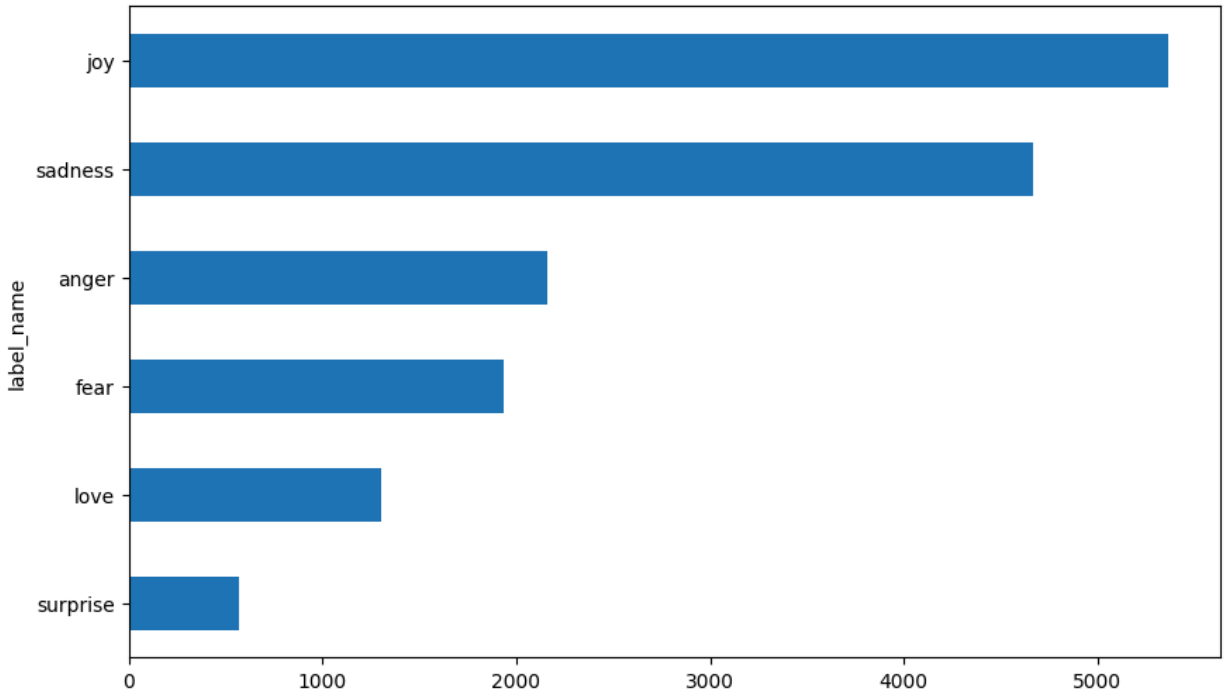
```
df['label'].value_counts()
```

label	count
1	5362
0	4666
3	2159
4	1937
2	1304
5	572

dtype: int64

```
import matplotlib.pyplot as plt
label_counts = df['label_name'].value_counts(ascending=True)
plt.figure(figsize=(10,6))
label_counts.plot.barh()
```

```
<Axes: ylabel='label_name'>
```



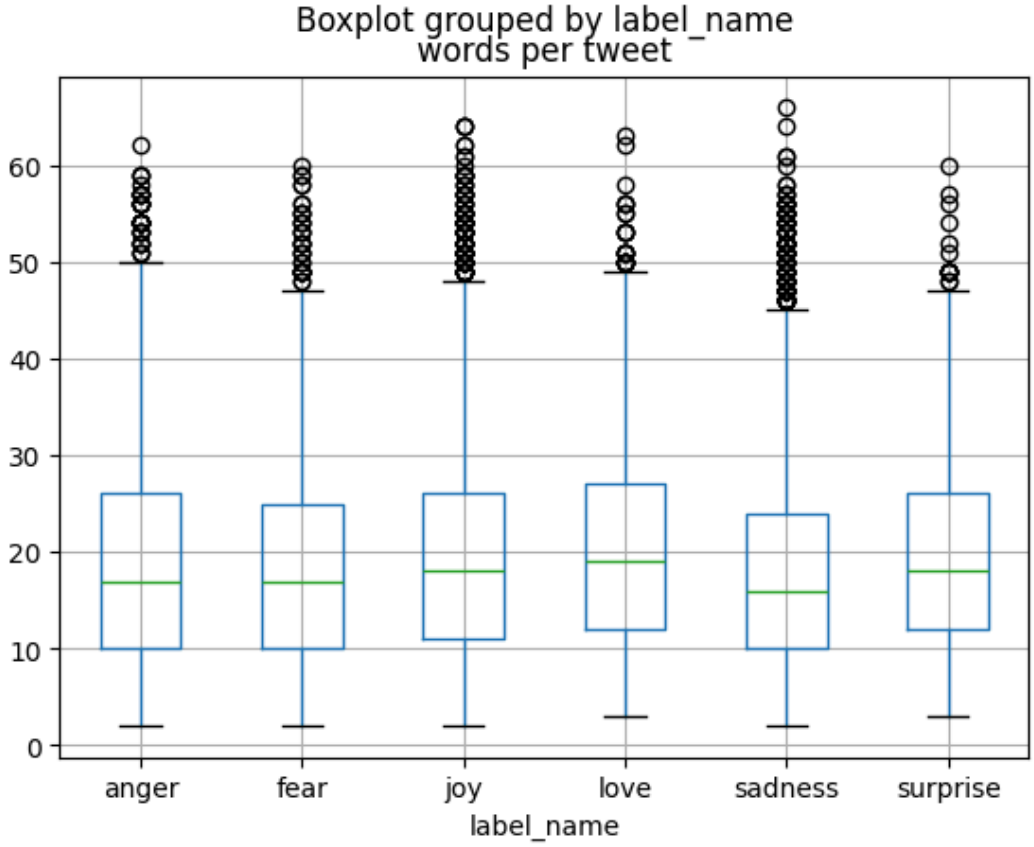

```
df['words per tweet'] = df['text'].str.split().apply(len)
df
```

	text	label	label_name	words per tweet
0	i didnt feel humiliated	0	sadness	4
1	i can go from feeling so hopeless to so damned...	0	sadness	21
2	im grabbing a minute to post i feel greedy wrong	3	anger	10
3	i am ever feeling nostalgic about the fireplac...	2	love	18
4	i am feeling grouchy	3	anger	4
...	...	...	...	...
15995	i just had a very brief time in the beanbag an...	0	sadness	24
15996	i am now turning and i feel pathetic that i am...	0	sadness	20
15997	i feel strong and good overall	1	joy	6
15998	i feel like this was such a rude comment and i...	3	anger	14

Next steps: Generate code with df New interactive sheet

```
df.boxplot("words per tweet", by="label_name")
```

```
<Axes: title={'center': 'words per tweet'}, xlabel='label_name'>
```




```
from transformers import AutoTokenizer
model_ckpt = 'bert-base-uncased'
Tokenizer = AutoTokenizer.from_pretrained(model_ckpt)



text = "I Love Machine Learning! Tokenizer is awesome"
encoded_text = Tokenizer(text)
print(encoded_text)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarnin
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public mo
  warnings.warn(
```

tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 2.32kB/s]

config.json: 100% 570/570 [00:00<00:00, 34.6kB/s]

vocab.txt: 100% 232k/232k [00:00<00:00, 3.61MB/s]

tokenizer.json: 100% 466k/466k [00:00<00:00, 3.55MB/s]

```
{'input_ids': [101, 1045, 2293, 3698, 4083, 999, 19204, 17629, 2003, 12476, 102], 'to
```

```
encoded_text
```

```
{'input_ids': [101, 1045, 2293, 3698, 4083, 999, 19204, 17629, 2003, 12476, 102],
'token_type_ids': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], 'attention_mask': [1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1]}
```

```
Tokenizer.vocab

len(Tokenizer.vocab), Tokenizer.vocab_size, Tokenizer.model_max_length
```

```
(30522, 30522, 512)
```

Train Test Split and Data Loader

```
import pandas as pd
url = "https://raw.githubusercontent.com/laxmimerit/All-CSV-ML-Data-Files-Download/m
df = pd.read_csv(url)
df['words per tweet'] = df['text'].str.split().apply(len)
display(df.head(16000))

from sklearn.model_selection import train_test_split

train, test = train_test_split(df, test_size=0.3, stratify=df['label_name'])
test, validation = train_test_split(test, test_size=0.5, stratify=test['label_name']

test.shape,test.shape, validation.shape
```

	text	label	label_name	words per tweet
0	i didnt feel humiliated	0	sadness	4
1	i can go from feeling so hopeless to so damned...	0	sadness	21
2	im grabbing a minute to post i feel greedy wrong	3	anger	10
3	i am ever feeling nostalgic about the fireplac...	2	love	18
4	i am feeling grouchy	3	anger	4
...	...	...	...	...
15995	i just had a very brief time in the beanbag an...	0	sadness	24
15996	i am now turning and i feel pathetic that i am...	0	sadness	20
15997	i feel strong and good overall	1	joy	6
15998	i feel like this was such a rude comment and i...	3	anger	14

```
from datasets import Dataset, DatasetDict

dataset = DatasetDict(
    {'train':Dataset.from_pandas(train, preserve_index=False),
     'test':Dataset.from_pandas(test, preserve_index=False),
     'validation':Dataset.from_pandas(validation, preserve_index=False)
     }
```

```
)
dataset
```

```
DatasetDict({
    train: Dataset({
        features: ['text', 'label', 'label_name', 'words per tweet'],
        num_rows: 11200
    })
    test: Dataset({
        features: ['text', 'label', 'label_name', 'words per tweet'],
        num_rows: 2400
    })
    validation: Dataset({
        features: ['text', 'label', 'label_name', 'words per tweet'],
        num_rows: 2400
    })
})
```

Tokenization of the sentiment Data

```
dataset['train'][0], dataset['train'][2]
```

```
({'text': 'i still feel its a little shaky at times and can move into the slightly
odd jades hair in particular seems prone to this but generally it works well with
spencers writing',
  'label': 4,
  'label_name': 'fear',
  'words per tweet': 32},
 {'text': 'i can t get past is that feeling when a friend walks out of your life and
you re unsure why that feeling of not being valued or important enough',
  'label': 4,
  'label_name': 'fear',
  'words per tweet': 30})
```

```
from os import truncate
def tokenize(batch):
    temp = Tokenizer(batch['text'], padding=True, truncation=True)
    return temp
print(tokenize(dataset['train'][:2]))
```

```
{'input_ids': [[101, 1045, 2145, 2514, 2049, 1037, 2210, 15311, 2012, 2335, 1998, 206
```

```
emotion_encoded = dataset.map(tokenize, batched=True, batch_size=None)
```

```
Map: 100%  11200/11200 [00:03<00:00, 3759.22 examples/s]
Map: 100%  2400/2400 [00:00<00:00, 2898.41 examples/s]
Map: 100%  2400/2400 [00:00<00:00, 2828.24 examples/s]
```

```
emotion_encoded
```

```
DatasetDict({
    train: Dataset({
        features: ['text', 'label', 'label_name', 'words per tweet', 'input_ids',
```

```
'token_type_ids', 'attention_mask'],
        num_rows: 11200
    })
    test: Dataset({
        features: ['text', 'label', 'label_name', 'words per tweet', 'input_ids',
'token_type_ids', 'attention_mask'],
        num_rows: 2400
    })
    validation: Dataset({
        features: ['text', 'label', 'label_name', 'words per tweet', 'input_ids',
'token_type_ids', 'attention_mask'],
        num_rows: 2400
    })
})
```

```
label2id =  {x['label_name']:x['label']for x in dataset['train']}
id2label = {v:k for k,v in label2id.items()}

label2id, id2label
```

```
({'fear': 4, 'joy': 1, 'anger': 3, 'sadness': 0, 'surprise': 5, 'love': 2},
 {4: 'fear', 1: 'joy', 3: 'anger', 0: 'sadness', 5: 'surprise', 2: 'love'})
```

Model Building

```
from transformers import AutoModel
import torch
```

```
model = AutoModel.from_pretrained(model_ckpt)
```

```
model.safetensors: 100%  440M/440M [00:08<00:00, 70.1MB/s]
```

```
model
```

```
BertModel(
  (embeddings): BertEmbeddings(
    (word_embeddings): Embedding(30522, 768, padding_idx=0)
    (position_embeddings): Embedding(512, 768)
    (token_type_embeddings): Embedding(2, 768)
    (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
    (dropout): Dropout(p=0.1, inplace=False)
  )
  (encoder): BertEncoder(
    (layer): ModuleList(
      (0-11): 12 x BertLayer(
        (attention): BertAttention(
          (self): BertSdpaSelfAttention(
            (query): Linear(in_features=768, out_features=768, bias=True)
            (key): Linear(in_features=768, out_features=768, bias=True)
            (value): Linear(in_features=768, out_features=768, bias=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
          (output): BertSelfOutput(
            (dense): Linear(in_features=768, out_features=768, bias=True)
            (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
            (dropout): Dropout(p=0.1, inplace=False)
```

```
            )
          )
          (intermediate): BertIntermediate(
            (dense): Linear(in_features=768, out_features=3072, bias=True)
            (intermediate_act_fn): GELUActivation()
          )
          (output): BertOutput(
            (dense): Linear(in_features=3072, out_features=768, bias=True)
            (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
        )
      )
    )
    (pooler): BertPooler(
      (dense): Linear(in_features=768, out_features=768, bias=True)
      (activation): Tanh()
    )
  )
```

```
model.config.id2label
model.config
```

```
BertConfig {
  "architectures": [
    "BertForMaskedLM"
  ],
  "attention_probs_dropout_prob": 0.1,
  "classifier_dropout": null,
  "dtype": "float32",
  "gradient_checkpointing": false,
  "hidden_act": "gelu",
  "hidden_dropout_prob": 0.1,
  "hidden_size": 768,
  "initializer_range": 0.02,
  "intermediate_size": 3072,
  "layer_norm_eps": 1e-12,
  "max_position_embeddings": 512,
  "model_type": "bert",
  "num_attention_heads": 12,
  "num_hidden_layers": 12,
  "pad_token_id": 0,
  "position_embedding_type": "absolute",
  "transformers_version": "4.56.2",
  "type_vocab_size": 2,
  "use_cache": true,
  "vocab_size": 30522
}
```

```
from transformers import AutoModelForSequenceClassification, AutoConfig
num_labels = len(label2id)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
config = AutoConfig.from_pretrained(model_ckpt, id2label=id2label, label2id=label2id
model =AutoModelForSequenceClassification.from_pretrained(model_ckpt, config=config)
```

```
Some weights of BertForSequenceClassification were not initialized from the model che
You should probably TRAIN this model on a down-stream task to be able to use it for p
```

```
model
```

```
BertForSequenceClassification(
  (bert): BertModel(
    (embeddings): BertEmbeddings(
      (word_embeddings): Embedding(30522, 768, padding_idx=0)
      (position_embeddings): Embedding(512, 768)
      (token_type_embeddings): Embedding(2, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (encoder): BertEncoder(
      (layer): ModuleList(
        (0-11): 12 x BertLayer(
          (attention): BertAttention(
            (self): BertSdpaSelfAttention(
              (query): Linear(in_features=768, out_features=768, bias=True)
              (key): Linear(in_features=768, out_features=768, bias=True)
              (value): Linear(in_features=768, out_features=768, bias=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
            (output): BertSelfOutput(
              (dense): Linear(in_features=768, out_features=768, bias=True)
              (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
          )
          (intermediate): BertIntermediate(
            (dense): Linear(in_features=768, out_features=3072, bias=True)
            (intermediate_act_fn): GELUActivation()
          )
          (output): BertOutput(
            (dense): Linear(in_features=3072, out_features=768, bias=True)
            (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
        )
      )
    )
    (pooler): BertPooler(
      (dense): Linear(in_features=768, out_features=768, bias=True)
      (activation): Tanh()
    )
  )
  (dropout): Dropout(p=0.1, inplace=False)
  (classifier): Linear(in_features=768, out_features=6, bias=True)
)
```

```
model.config
```

```
BertConfig {
  "architectures": [
    "BertForMaskedLM"
  ],
  "attention_probs_dropout_prob": 0.1,
  "classifier_dropout": null,
  "dtype": "float32",
  "gradient_checkpointing": false,
  "hidden_act": "gelu",
  "hidden_dropout_prob": 0.1,
  "hidden_size": 768,
  "id2label": {
    "0": "sadness",
    "1": "joy",
```

```
    "2": "love",
    "3": "anger",
    "4": "fear",
    "5": "surprise"
  },
  "initializer_range": 0.02,
  "intermediate_size": 3072,
  "label2id": {
    "anger": 3,
    "fear": 4,
    "joy": 1,
    "love": 2,
    "sadness": 0,
    "surprise": 5
  },
  "layer_norm_eps": 1e-12,
  "max_position_embeddings": 512,
  "model_type": "bert",
  "num_attention_heads": 12,
  "num_hidden_layers": 12,
  "pad_token_id": 0,
  "position_embedding_type": "absolute",
  "transformers_version": "4.56.2",
  "type_vocab_size": 2,
  "use_cache": true,
  "vocab_size": 30522
}
```

```
from transformers import TrainingArguments

batch_size = 64
training_dir = "bert_base_train_dir"

training_args = TrainingArguments( output_dir=training_dir,
                                   overwrite_output_dir = True,
                                    num_train_epochs=2,
                                    learning_rate=2e-5,
                                    per_device_train_batch_size = batch_size,
                                    per_device_eval_batch_size = batch_size,
                                    weight_decay = 0.01,
                                    eval_strategy = 'epoch',
                                    disable_tqdm = False,
                                    report_to = 'none'

)
```

```
## build compute matrics functions
!pip install evaluate
```

```
Collecting evaluate
  Downloading evaluate-0.4.6-py3-none-any.whl.metadata (9.5 kB)
Requirement already satisfied: datasets>=2.0.0 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: dill in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: requests>=2.19.0 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: tqdm>=4.62.1 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: multiprocess in /usr/local/lib/python3.12/dist-package
```

```
Requirement already satisfied: fsspec>=2021.05.0 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: huggingface-hub>=0.7.0 in /usr/local/lib/python3.12/di
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in /usr/local/lib/python3.1
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.1
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/python3.12/dist
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/di
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/d
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (f
Downloading evaluate-0.4.6-py3-none-any.whl (84 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 84.1/84.1 kB 3.3 MB/s eta 0:00:00
Installing collected packages: evaluate
Successfully installed evaluate-0.4.6
```

```
import evaluate
import numpy as np

accuracy = evaluate.load("accuracy")


def compute_metrics_evaluate(eval_pred):
    predictions, labels = eval_pred
    predictions = np.argmax(predictions, axis=1)
    return accuracy.compute(predictions=predictions, references=labels)
```

```
Downloading builder script:       4.20k/? [00:00<00:00, 83.5kB/s]
```

```
from sklearn.metrics import accuracy_score, f1_score

def compute_metrics(pred):
    labels = pred.label_ids
    preds = pred.predictions.argmax(-1)
    f1 = f1_score(labels, preds, average="weighted")
    acc = accuracy_score(labels, preds)
    return {"accracy" : acc, "f1": f1}
```

Build Model and Trainer

```
from transformers import Trainer
```

```
trainer = Trainer(
    model=model,
    args=training_args,
    compute_metrics=compute_metrics,
    train_dataset=emotion_encoded['train'],
    eval_dataset=emotion_encoded['validation'])
```

```
emotion_encoded
```

```
DatasetDict({
    train: Dataset({
        features: ['text', 'label', 'label_name', 'words per tweet', 'input_ids',
'token_type_ids', 'attention_mask'],
        num_rows: 11200
    })
    test: Dataset({
        features: ['text', 'label', 'label_name', 'words per tweet', 'input_ids',
'token_type_ids', 'attention_mask'],
        num_rows: 2400
    })
    validation: Dataset({
        features: ['text', 'label', 'label_name', 'words per tweet', 'input_ids',
'token_type_ids', 'attention_mask'],
        num_rows: 2400
    })
})
```

```
trainer.train()
```

[350/350 06:18, Epoch 2/2]

Epoch	Training Loss	Validation Loss	Accracy	F1
1	No log	0.178686	0.931250	0.931428
2	No log	0.179866	0.932917	0.932956

```
TrainOutput(global_step=350, training_loss=0.10005129132952009, metrics=
{'train_runtime': 379.7153, 'train_samples_per_second': 58.992,
'train_steps_per_second': 0.922, 'total_flos': 1001502421516800.0, 'train_loss':
0.10005129132952009, 'epoch': 2.0})
```

Model Evaluation

```
preds_output =trainer.predict(emotion_encoded['test'])
preds_output.metrics
```

```
{'test_loss': 0.30215567350387573,
 'test_accracy': 0.9029166666666667,
 'test_f1': 0.9024371456107466,
 'test_runtime': 9.2578,
 'test_samples_per_second': 259.24,
 'test_steps_per_second': 4.105}
```

```
y_preds = np.argmax(preds_output.predictions, axis=1)
y_true = emotion_encoded['test']['label']
```

```
emotion_encoded['test']
```

```
Dataset({
    features: ['text', 'label', 'label_name', 'words per tweet', 'input_ids',
'token_type_ids', 'attention_mask'],
    num_rows: 2400
})
```

```
from sklearn.metrics import classification_report
print(classification_report(y_true, y_preds))
```

```
              precision    recall  f1-score   support

           0       0.92      0.95      0.94       700
           1       0.95      0.90      0.93       804
           2       0.79      0.82      0.80       195
           3       0.90      0.93      0.92       324
           4       0.84      0.89      0.86       291
           5       0.81      0.63      0.71        86

    accuracy                           0.90      2400
   macro avg       0.87      0.85      0.86      2400
weighted avg       0.90      0.90      0.90      2400
```

```
label2id
```

```
{'fear': 4, 'joy': 1, 'anger': 3, 'sadness': 0, 'surprise': 5, 'love': 2}
```

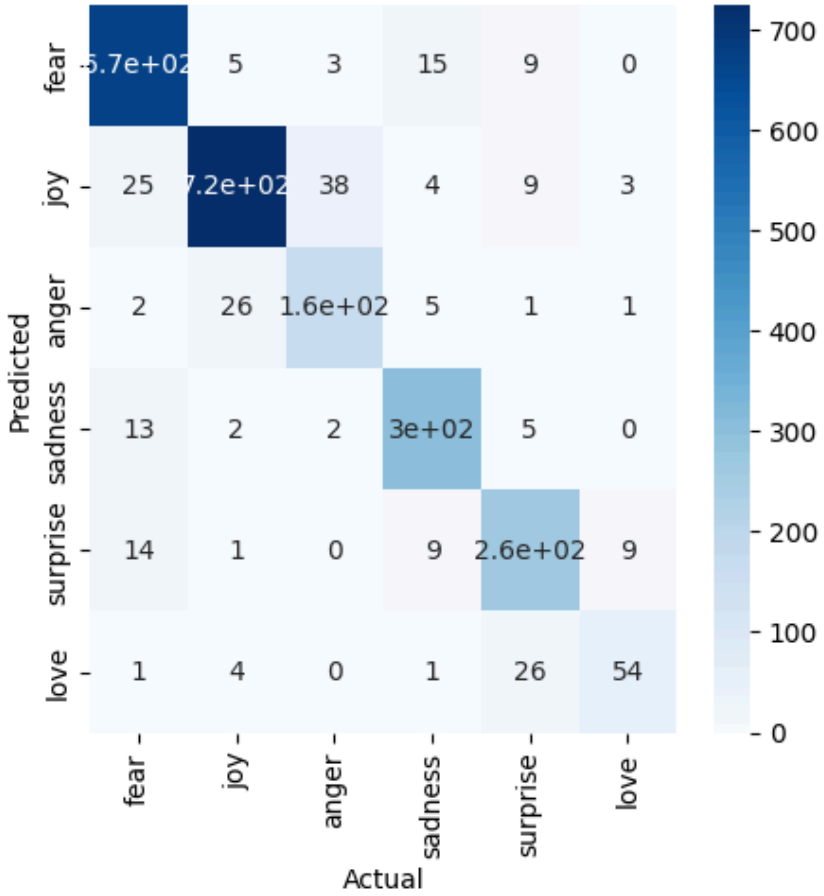
```
## plot confusion matrix
import seaborn as sns
import sklearn.metrics as metrics
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix


cm = confusion_matrix(y_true, y_preds)
cm
```

```
array([[668,   5,   3,  15,   9,   0],
       [ 25, 725,  38,   4,   9,   3],
       [  2,  26, 160,   5,   1,   1],
       [ 13,   2,   2, 302,   5,   0],
       [ 14,   1,   0,   9, 258,   9],
       [  1,   4,   0,   1,  26,  54]])
```

```
plt.figure(figsize=(5,5))
sns.heatmap(cm, annot=True, xticklabels=label2id.keys(), yticklabels=label2id.keys()
plt.xlabel("Actual")
plt.ylabel("Predicted")
```

```
Text(33.222222222222214, 0.5, 'Predicted')
```



Build Prediction Function and Store Model

```
text = "I am super happy today. I got it done. finally!!"

def get_prediction(text):
    input_encoded = Tokenizer(text, return_tensors='pt').to(device)


    with torch.no_grad():
        output = model(**input_encoded)

    logits = output.logits

    pred = torch.argmax(logits, dim=1).item()
    return pred, id2label[pred]

get_prediction(text)
```

```
(1, 'joy')
```

```
trainer.save_model("bert-base-uncased-sentiment-model")
```

```
####use pipeline for prediction[M-2]
from transformers import pipeline

classifier = pipeline('text-classification', model='bert-base-uncased-sentiment-model'

classifier([text, 'hello, how are you', 'I am feeling not good'])
```

```
classifier([text, "hello, how are you ", "I am feeling not good "])
```